

Will My Policy Pay?

Health insurance fine print, explained before you need it: every limit quoted from the insurer's own document, and what a hospital bill would actually pay.

Akshay Dhar · akshaykumardhar14@gmail.com · github.com/AKSHAYKUMARDHAR/Will-My-Policy-Pay

MY ROLE

Solo: research, PRD, labelling, evaluation design and the build, with Claude Code.

WHEN

Oct 2026

STACK

Python, FastAPI, Gemini (reads PDFs), pypdf, vanilla JS, Docker

23

real policy documents from 15 insurers, each frozen in git before use

8.2% → 1.2%

wrong facts on the dev set once code checks every quote (same model answers)

25 / 25

hand-worked hospital bills matched by the simulator

Blocked twice

by my own release gate: wrong facts above the 2% bar on unseen policies, in both release runs

The problem

In a 2025 LocalCircles survey, only 25% of policyholders who had claimed said their claims were fully approved; 36% had a claim rejected and 33% had one only partly paid. Insurers processed 3.26 crore health claims in 2024-25 and repudiated 8%, and IRDAI told Parliament it doesn't record why.

Much of the money is lost in a few lines of fine print people never read. With a room-rent cap of 1% of the sum insured, a ₹5 lakh policy pays ₹5,000 a day for the room; take an ₹8,000 room and most doctor and surgery charges are cut in the same proportion. On a knee replacement that turns ₹15,000 of extra rent into ₹60,000 out of pocket.

Who it's for

- **The family guardian** checking a parent's older policy before a hospital visit.
- **The new buyer** comparing two policies, now cheaper since GST came off individual health premiums in September 2025.
- **The patient before a planned surgery** deciding which room to take.

What I built

A free web tool. Pick a policy (or upload its PDF) and get a fine-print card of 17 terms: room and ICU limits, co-payments, deductible, cataract and modern-treatment caps, waiting periods, maternity, bonus and restoration. Every value shows the exact words and page from the insurer's document. A bill simulator then shows line by line what a hospital bill would pay, and a guide covers what to do if a claim is cut, with a grievance letter.

Policy PDF

insurer's wording or summary

Model reads it twice

17 terms, each with a quote and page

Code checks

reads agree, quote on the page, numbers in the quote

Card and simulator

fixed explanations; arithmetic in code

Will My Policy Pay? Check a policy Bill simulator If a claim is cut

Your health policy's fine print, in plain English, before you need it.

Room-rent limits, co-payments, sub-limits and waiting periods decide what a claim actually pays. See yours, with the exact words and page from your insurer's own document, then see what a hospital bill would pay.

Nothing here sells insurance or recommends a policy. Every number is quoted from the document; if we can't prove it, we say so.

1. Check a policy

Pick a policy we've already read

New India Premier Mediclaim Policy New India Assurance - CIS	Health Care Supreme Bajaj Allianz General Insurance - policy wording
MediCare Tata AIG General Insurance - policy wording	National Parivar Mediclaim Plus Policy National Insurance - prospectus
Star Comprehensive Insurance Policy Star Health and Allied Insurance - policy wording	

POLICY: National Parivar Mediclaim Plus Policy INSURER: National Insurance READ FOR: Sum insured ₹6 lakh, age 48

DOCUMENT: prospectus - RTDLTP2151V022821 - 24 pages

Watch out for

1. **Room rent limit:** Up to ₹6,000 a day (1% of your cover)
2. **Cataract limit:** Up to ₹60,000 per eye (15% of your cover, at most ₹60,000)
3. **ICU limit:** Up to ₹12,000 a day (2% of your cover)

11 terms confirmed from the document, 4 not in it, 2 to check yourself. Open "Show the clause" to see the exact words.

What can cut your claim

Room rent limit Up to ₹6,000 a day (1% of your cover) **WATCH OUT**
Pick a costlier room and the extra rent is yours to pay, and most doctor and surgery charges are cut in the same proportion. Medicines, implants, tests and ICU charges are never cut this way.
▼ Show the clause
Room - Up to 1% of SI or actual, whichever is lower
Page 20 of the insurer's document
[This looks wrong](#)

ICU limit Up to ₹12,000 a day (2% of your cover) **WATCH OUT**
ICU charges above this are yours to pay.
► Show the clause
[This looks wrong](#)

Co-payment Check this one yourself CHECK YOURSELF
The words we found exempt some people (by age or city zone) rather than say there is none for everyone.
▼ What the two readings found
Insured paying premium as per Zone I can avail treatment in Zone I, Zone II, Zone III and Zone IV without copayment
Page 14
Insured paying premium as per Zone I can avail treatment in Zone I, Zone II, Zone III and Zone IV without copayment
Page 14

Hospital bill: ₹3,51,000 Policy pays: ₹3,05,000 You pay: ₹46,000

Choose a room within your limit (₹6,000 a day) and you pay ₹40,000 less.

ITEM	BILLED	POLICY PAYS	YOU PAY
Room, 5 days at ₹8,000 Your room limit is ₹6,000 a day	₹40,000	₹30,000	₹10,000
Doctor, surgeon and OT charges (room-linked) Proportionate deduction: paid at 75.0%, the same share as your room rent	₹1,20,000	₹90,000	₹30,000
Medicines and consumables Never cut in proportion to the room rent (IRDAI/HLT/REG/CIR/151/06/2020)	₹60,000	₹60,000	₹0
Diagnostics and tests Never cut in proportion to the room rent (IRDAI/HLT/REG/CIR/151/06/2020)	₹20,000	₹20,000	₹0
Implants and devices Never cut in proportion to the room rent (IRDAI/HLT/REG/CIR/151/06/2020)	₹1,00,000	₹1,00,000	₹0
Before admission and after discharge	₹5,000	₹5,000	₹0
Non-medical items (gloves, attendant food, toiletries...) Not payable under IRDAI's list of non-medical items, unless you have a consumables cover	₹6,000	₹0	₹6,000

No deductible was confirmed from the document, so this assumes there is none. Check your policy schedule.

No co-payment was confirmed from the document, so this assumes there is none. Check your policy schedule.

An estimate from the limits on the card; the claim depends on the full policy, the hospital's bill and the insurer's assessment.

Also in the product: share links that open a card or a sample bill, a "This looks wrong" button on every term, feedback questions that measure the PRD's goals, and an event log that stores no document text. Uploaded PDFs are read and thrown away. When the free model quota runs out, uploads say so plainly and the ready-made cards keep working.

Product decisions

- **The model reads; code checks, explains and calculates.** Gemini only extracts values with quotes. Every explanation a user reads is written once in code, and the bill simulator is deterministic, tested against 25 hand-worked bills.
- **Proof or nothing.** A term is shown only when two independent reads agree, the quote is found on the cited page and contains the value's numbers. Otherwise the card says "not found" or "check this yourself", with both quotes.
- **"No limit" needs the words.** A summary that lists "Expenses incurred towards Ambulance" doesn't say there is no cap; the policy schedule might set one. Saying "no limit" wrongly is the error this product exists to prevent.
- **IRDAI's rules in code.** The proportionate deduction never touches medicines, implants, tests or ICU charges (IRDAI's 2020 circular), and the simulator cites it on each line.
- **Neutral by design.** No recommendations, rankings or commissions: selling insurance is licensed activity in India.
- **English only, written for a first-time reader.**

How I tested it

This time the test data is real. I collected **14 public policy documents from 10 insurers** (wordings, customer information sheets and prospectuses) and froze them in git with their hashes **before writing any prompt**. Then I wrote the answer keys by hand from the documents: 85 facts for 5 dev documents and 153 for 9 held-out documents, each with the exact quote and page, checked against the PDFs by a script. The main labelling rule is "only what the document says": if a summary doesn't state a limit, the answer is "not stated", because the policy schedule can still set one.

Every answer is scored **correct**, **wrong** (a value shown that the document doesn't support: the error that costs money) or **withheld** ("not found" or "check this yourself" where the document does state a value). I ran the same model answers with and without the checks, to measure what the checks themselves do.

Dev run (85 facts)	Correct	Wrong	Withheld
Prompt v1 · 2 reads, no checks	88%	8.2%	4%
Prompt v1 · 2 reads + checks	74%	1.2%	25%
Prompt v2 · 2 reads + checks (4 of 5 documents)	91%	1.5%	7%
Prompt v3 · smaller model (gemini-3.5-flash-lite)	80%	8.2%	12%
Prompt v3 · 2 reads + checks	92%	3.5%	5%
Prompt v3 + "none for everyone" checks · frozen	91%	0%	9%

Then I froze the configuration (prompt version 3 on gemini-3.5-flash, two reads, every check) and ran the 9 held-out documents once.

Held-out run (152 facts)	Correct	Wrong	Withheld
1 read, no checks	90.8%	3.9%	5.3%
2 reads, no checks	93.4%	3.9%	2.6%
2 reads + checks · what ships	91.4%	3.3%	5.3%

On review, 3 of release run 1's 5 wrong facts were mistakes in my answer key (more below), so I corrected the rules, moved those documents to development, and froze 9 fresh documents from 8 insurers for release run 2: 141 scored facts, 61 of them "not stated", labelled before any model read them, on the unchanged system. Google's servers failed the first document with "high demand" errors whose retries used up the day's free requests, so after 4 documents the result was already settled:

Release run 2, first 4 of 9 documents (67 facts)	Correct	Wrong	Withheld
1 read, no checks	86.6%	4.5%	9.0%
2 reads, no checks	89.6%	4.5%	6.0%
2 reads + checks · what ships	80.6%	4.5%	14.9%

Release gate

- ✓ Bill simulator matches all 25 hand-worked bills: 25 of 25
- ✗ Run 1, wrong facts $\leq 2\%$: 5 of 152 (3.3%)
- ✓ Run 1, correct $\geq 85\%$: 139 of 152 (91.4%)
- ✗ Run 2, wrong facts $\leq 2\%$: 3 in the first 4 documents, against 2 allowed for all 9
- ✗ Run 2, correct $\geq 85\%$: 80.6% on the first 4 documents
- ✓ Every value shown is quoted and found on its cited page: all of them

What failed, and what's next

The checks cut wrong answers sevenfold, and at first they withheld a quarter of the card. Version 1 of the prompt let the model quote a heading or a definition instead of the line that sets the limit, so correct values failed verification. Version 2 asks for the words that set each value and lets the model join two passages with "...", each checked on its own; withheld facts fell from 25% to 7%.

The worst early error called ambulance cover "up to your sum insured" from a line that only said "Expenses incurred towards Ambulance". A "no limit" answer now needs words that say so. Version 3 still showed 3 wrong facts on the full dev set: two read "no co-payment" from sentences that only exempt some people (by entry age or city zone), and one took a ceiling shared by several covers as AYUSH's own limit. Two more code checks send such quotes to "check this yourself", which took wrong facts to zero on the same answers and cost one correct one. The smaller, cheaper models failed the bar: both read "1% of the sum insured or actual, whichever is lower" as a choice of two limits and called a premium discount a bonus. The model that passes allows 20 free requests a day, so ready-made cards are built from the evaluation's own readings and uploads are capped.

Release run 1 failed my own gate. On 9 policies it had never seen, 139 of 152 facts were right and 5 were wrong: 3.3% against a 2% bar. The score stands, because held-out labels never change after a run. On review, though, none of the 5 is a clear model error. Two joined a separate 90-day wait for diabetes, hypertension and heart conditions to the 24-month specific-illness clause: wider than my answer key, and real for a buyer with diabetes. Three were mistakes in my answer key, including HDFC's AYUSH limit, which I first reported as the model's worst error. The model had quoted the AYUSH row of the insurer's plan table ("Covered upto sum insured"), and my key had missed that table.

So version 2 changed the answer-key rules, not the system, and **release run 2 failed the gate on real errors.** IFFCO Tokio's reinstatement, bought at the time of a claim, was shown as a 100% refill of cover. National's 20% co-payment in non-network hospitals was read as "no co-payment": the exemption mistake the dev checks catch for age and city zone, but not for "non-network". And Liberty's 90-day wait for listed critical illnesses was left out of the waiting periods; my key had its own miss there (a 24-month list on page 14), which stays logged rather than fixed. The other 5 documents are read the next day, from the same frozen system.

Next: an exemption check that covers network status, a check that an "optional" or "may opt" clause can't become a base-plan value, and then a third set of unseen documents, since a fix tuned on run 2 has to be tested on documents it hasn't seen. Later: a second labeller, the policy schedule as a second upload (it sets the limits a wording leaves open), and scanned PDFs, which the tool currently turns away.

What I learned

- Verification is the product. The same two reads showed 7 wrong facts without the checks and 1 with them.
- Check the answer key before blaming the model, and then check the model again. Three of release run 1's five "wrong" facts were my labelling mistakes, and the fix I first drafted for the AYUSH "error" would have cost 4 correct answers and caught nothing. Release run 2's three errors were real.
- Outages cost quota. Google's "high demand" errors counted against the free daily limit, so the retry policy is part of the evaluation design: a document the provider never served is now left out of the scores, not counted as withheld.
- The labeller makes mistakes too. The model found a bonus clause I had missed (Bajaj, page 22); I corrected the dev key and logged it. Held-out labels never change after a run.
- Real documents beat hand-written test data: every hard case (tables, two-column pages, "whichever is lower") came from an insurer's own PDF.
- Free tiers shape evaluation plans: 20 requests a day decided the order of every run.

The 14 documents are real, public policy documents; the answer keys are mine, written from the documents before any model read them. Every number comes from a logged run in the repo.